



Práctica 3

APRENDIZAJE POR REFUERZO MULTIAGENTE EN EL ENTORNO
POGEMA

Por:

Víctor Ramírez Arimaha, Marcel Alabart Benoit y Adrià Cebrián Ruiz

Facultad Informática de Barcelona (FIB)
Sistemas Inteligentes Distribuidos (SID)

Índice

1. Introducción	2
2. Entorno de Pogema	2
3. Explicación de los algoritmos	3
3.1. JAL-GT	3
3.2. IQL	3
3.3. JALGTNN	4
4. Experimentos	5
4.1. Metodología del estudio	5
4.2. JAL-GT	7
4.2.1. Minimax	7
4.2.2. Óptimo de Pareto	9
4.2.3. Welfare	11
4.2.4. Equilibrios de Nash	13
4.2.5. Otras métricas	15
4.3. IQL	18
4.3.1. Distribución de recompensas	18
4.3.2. Interpretación de las políticas	19
4.3.3. Generalización de los parámetros	20
4.4. JALGTNN	21
4.4.1. Minimax	21
4.4.2. Óptimo de Pareto	23
4.4.3. Welfare	25
4.4.4. Equilibrios de Nash	27
4.5. Parámetros óptimos	29
5. Conclusión	30

1. Introducción

En este trabajo se estudia el comportamiento de tres algoritmos de aprendizaje por refuerzo multiagente en el entorno POGEMA. Los algoritmos analizados son: JAL-GT (Joint Action Learning with Game Theory), su variante con redes neuronales (JALGTNN) y IQL (Independent Q-Learning). El objetivo principal es comparar su rendimiento utilizando diferentes conceptos de solución (Minimax, Pareto, Welfare y Nash) y analizar cómo afectan los parámetros a su convergencia y comportamiento.

2. Entorno de Pogema

POGEMA (Partially Observable Multi-Agent Pathfinding), es un entorno que, como su propio nombre indica, es multiagente, además de parcialmente observable.

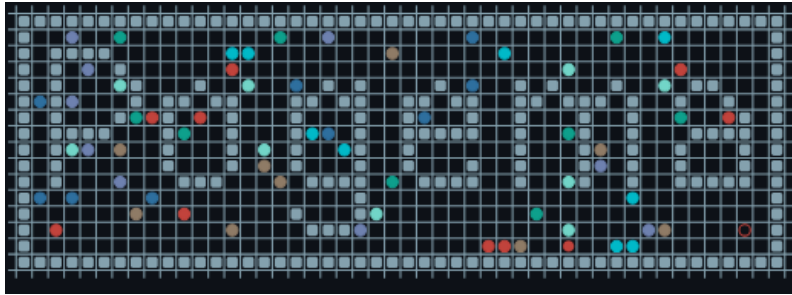


Figura 1: Ejemplo de mapa del entorno de *Pogema*

Todos los agentes tienen un campo de visión limitado a partir del cual observan el entorno, con el objetivo de llegar a un punto que les ha sido asignado en la inicialización del entorno, del cual solo tienen la dirección general.



Figura 2: Campo de visión de un agente en Pogema

Como podemos ver en la figura 2,

Los agentes han de colaborar y trabajar entre ellos con tal de no obstruirse, ya que es

un juego con recompensa colectiva, de manera que hay cierto incentivo de coordinarse y entender al resto de agentes.

3. Explicación de los algoritmos

3.1. JAL-GT

JAL-GT es un algoritmo de diferencias temporales que aproxima los valores de estado-acción de todos los agentes usando Q-learning. Las recompensas y la política se basan en los valores de Q, a la que luego se aplica el concepto de solución en cuestión (Minimax, Pareto, Nash o Welfare).

Algorithm 7 Joint-action learning with game theory (JAL-GT)

```

// Algorithm controls agent i
1: Initialize:  $Q_j(s, a) = 0$  for all  $j \in I$  and  $s \in S, a \in A$ 
2: Repeat for every episode:
3:   for  $t = 0, 1, 2, \dots$  do
4:     Observe current state  $s^t$ 
5:     With probability  $\epsilon$ : choose random action  $a_i^t$ 
6:     Otherwise: solve  $\Gamma_{s^t}$  to get policies  $(\pi_1, \dots, \pi_n)$ , then sample action  $a_i^t \sim \pi_i$ 
7:     Observe joint action  $a^t = (a_1^t, \dots, a_n^t)$ , rewards  $r_1^t, \dots, r_n^t$ , next state  $s^{t+1}$ 
8:     for all  $j \in I$  do
9:        $Q_j(s^t, a^t) \leftarrow Q_j(s^t, a^t) + \alpha [r_j^t + \gamma \text{Value}_j(\Gamma_{s^{t+1}}) - Q_j(s^t, a^t)]$ 

```

Figura 3: Pseudocódigo de JALGT extraído del notebook de **SID**

De manera que Γ se define como el concepto de juego, que es una matriz formada por los valores de Q para cada acción conjunta y agentes.

$$\Gamma_{s,i} = \begin{bmatrix} Q_i(s, a_{i,1}, a_{j,1}) & Q_i(s, a_{i,1}, a_{j,2}) & Q_i(s, a_{i,1}, a_{j,3}) \\ Q_i(s, a_{i,2}, a_{j,1}) & Q_i(s, a_{i,2}, a_{j,2}) & Q_i(s, a_{i,2}, a_{j,3}) \\ Q_i(s, a_{i,3}, a_{j,1}) & Q_i(s, a_{i,3}, a_{j,2}) & Q_i(s, a_{i,3}, a_{j,3}) \end{bmatrix}$$

Figura 4: Concepto de juego extraído del notebook de **SID**

3.2. IQL

Independent Q-Learning, como su nombre indica, consiste en entrenar cada agente individualmente con *Q-Learning*, tratando al resto de agentes como si se trataran de

elementos dinámicos en el entorno.

Esto hace que el entorno sea dinámico desde la perspectiva de cada agente. Además, no utiliza conceptos de solución, ya que cada agente optimiza su propia recompensa individual sin considerar las acciones conjuntas.

3.3. JALGTNN

JALGTNN es una variante de *JAL-GT* que en vez de aprender y guardarse los valores de Q , entrena a una red neuronal, generando una función de estimación de los valores de Q que, cuando se hace bien, suele ser muy certera y realista a los valores reales.

Esta implementación tiene una mejor escalabilidad para un mayor número de agentes, que para el algoritmo de *JAL-GT* original, equivale a añadir una nueva dimensión a la matriz de Q .

4. Experimentos

4.1. Metodología del estudio

La recolección de experimentos se ha llevado a cabo utilizando la librería de optimización de hiperparámetros llamada *Optuna*. Esta librería permite optimizar la exploración del espacio de hiperparámetros, debido a que automáticamente se encarga de "dirigirse" a zonas que maximicen un objetivo, en este caso la **recompensa colectiva media**. Mediante el sampler *Tree-structured Parzen Estimator* que según la documentación [de optuna](#):^{En} cada ensayo, para cada parámetro, TPE ajusta un Modelo de Mezcla Gaussiana (MGM) $\mathbf{x}$ al conjunto de valores de los parámetros asociados con los mejores valores objetivo, y otro MGM $\mathbf{g}(\mathbf{x})$ a los valores restantes. Selecciona el valor del parámetro $\mathbf{x}$ que maximiza la razón $1(\mathbf{x})/\mathbf{g}(\mathbf{x})$. Esto implica una importante mejora en la recolección de datos respecto a *Random Search*.

Optuna permite, para cada hiperparámetro, proponer un rango de valores. Para la elección de estos rangos de valores, se decidió que cada combinación de algoritmos y concepto de solución tuviese uno propio. Es decir que se estudiarán los mejores hiperparámetros para cada combinación de algoritmo y solución de concepto individualmente. Antes de llevar a cabo los experimentos finales, se hizo un poco de *fine tuning*, observando para cada ejecución si los parámetros seleccionados se acercaban a alguno de sus límites inferiores o superiores, y para los casos en que así fuese, se extendieron los valores.

En el análisis de los experimentos, aunque *Optuna* se haya optimizado principalmente con el resultado de la recompensa colectiva, se analizará también el tiempo de entrenamiento, así como la capacidad que tienen las diferentes combinaciones de parámetros de generalizarse para funcionar en entornos distintos.

Se han recolectado cuatro muestras (*trials*) en paralelo por cada combinación de hiperparámetros y optimizado los algoritmos con funciones de compilación *Just in time* con *numba*.

Los parámetros optimizados incluyen:

- Tasa de aprendizaje (α)
- Factor de descuento (γ)
- Valores de ϵ (exploración aleatoria máxima y mínima)

- Tamaño de la red neuronal (para JALGTNN)
- Longitud de episodios
- Cantidad de episodios

No incluyen, por desgracia, la representación del estado ni las recompensas del entorno como parámetros. En caso de disponer de más tiempo se hubiesen probado posibilidades como:

- Un bit por cada dirección (también en diagonal) para el objetivo, obstáculos y agentes.
- Un bit por cada dirección (sin diagonales, extendidas en sentido horario) para el objetivo, obstáculos y agentes.
- Eliminar la penalización (recompensa negativa) por paso.
- Reducir o aumentar la recompensa del objetivo.

4.2. JAL-GT

En este subapartado se hará el estudio para la optimización de los hiperparámetros en *JAL-GT*, para cada uno de los conceptos de solución. Una vez encontrado un conjunto de hiperparámetros que demuestren buenos resultados en un entorno simple con una densidad de objetos de 0.1, un tamaño de 4 y 2 agentes, probaremos con los mismos hiperparámetros entornos más complejos con 3 y 4 agentes además de más densidad de obstáculos y mayor tamaño.

4.2.1. Minimax

El concepto Minimax busca maximizar la recompensa mínima que puede obtener un agente.

4.2.1.1 Distribución de recompensas

En la siguiente figura se puede observar cómo los distintos parámetros afectan a los valores de las recompensas resultantes.

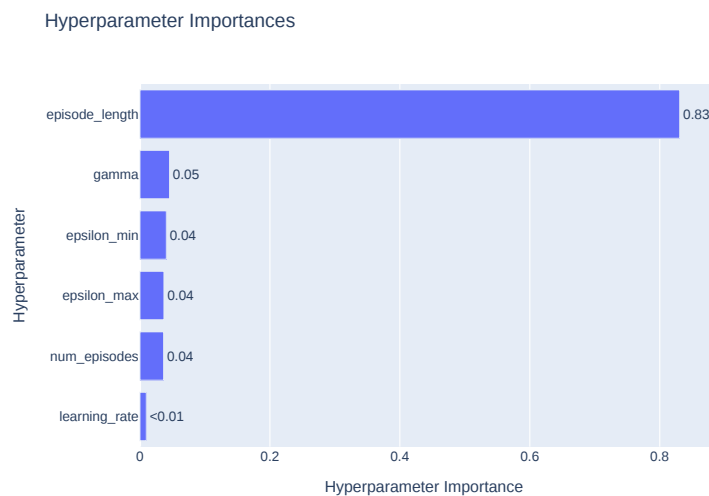


Figura 5: Parámetros más importantes Minimax de *Optuna*

Inesperadamente, el parámetro con más efecto (totalmente predominante) es la cantidad de pasos máxima por episodio. Nuestra hipótesis es que se debe a que, cuando la cantidad de pasos no es suficiente para alcanzar el objetivo, el cálculo de *MiniMax* deja de tener sentido ya que intenta minimizar el máximo de nada. Y, en cuanto la cantidad de pasos se vuelve suficiente, comienza a actuar con cierto sentido, de todas formas, como veremos en

los resultados, minimizar las recompensas del resto de agentes no es una estrategia ideal para este entorno. A continuación, se hará un estudio para los parámetros más importantes según *Optuna*:

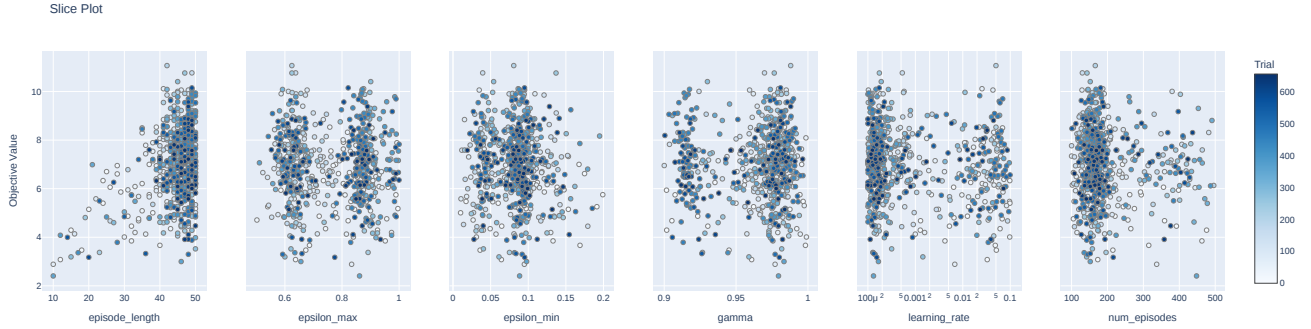


Figura 6: Resultados parámetros importantes Minimax

- **Episode length:** queda claro que, dentro del rango determinado, cuanto más grande sea el límite de pasos más alta se vuelve la recompensa colectiva, ya que la gran mayoría de intentos se centran en el extremo mayor, sobre todo acercándose al final del estudio, después del *trial* 500.
- **Epsilon max:** los mejores resultados para la *epsilon max* se divide en dos distribuciones, una ligeramente por encima de 0.6, y la otra aproximadamente 0.85.
- **Epsilon min:** los mejores resultados para la *epsilon min* se encuentran entre 0.1 y 0.15.
- **Gamma:** los mejores resultados para la *Gamma*, de nuevo, se dividen en dos franjas en los valores 0.92 y 0.98.
- **Learning Rate:** los mejores resultados para la *Learning Rate* parecen acumularse a la baja, aunque parece que durante algunos intentos se exploró la opción opuesta.
- **Num episodes:** los mejores resultados para la *Num episodes* están claramente centrados en los 150 episodios.

El método de minimax no es escalable a entornos con más de dos agentes debido a la propia implementación de minimax.

4.2.2. Óptimo de Pareto

Busca soluciones donde ningún agente puede mejorar su recompensa sin empeorar la de otro. Implementado como `ParetoSolutionConcept` en `solution_concepts.py`.

4.2.2.1 Distribución de recompensas

En la siguiente figura se puede observar cómo los distintos parámetros afectan a los valores de las recompensas resultantes. En caso del concepto de solución de *Óptimo de Pareto* las importancias están bastante equilibradas. Se podría argumentar que es debido a que no existe ningún cambio de fase como en algunos otros conceptos y por lo tanto todos los hiperparámetros tienen un efecto parecido en la recompensa colectiva.

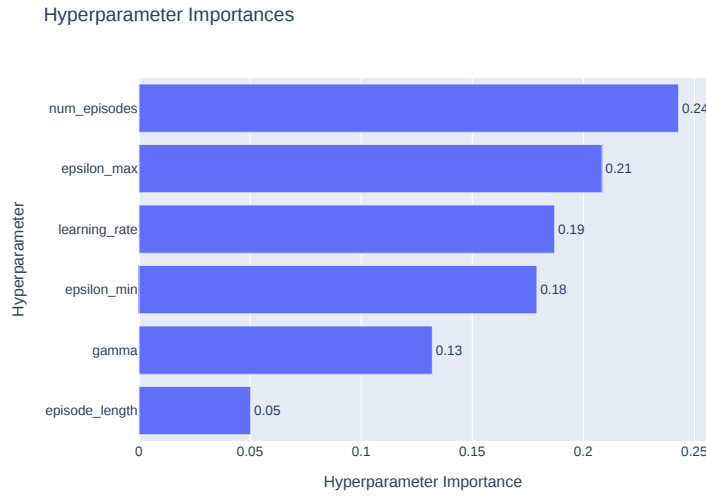


Figura 7: Parametros más importantes Pareto de *Optuna*

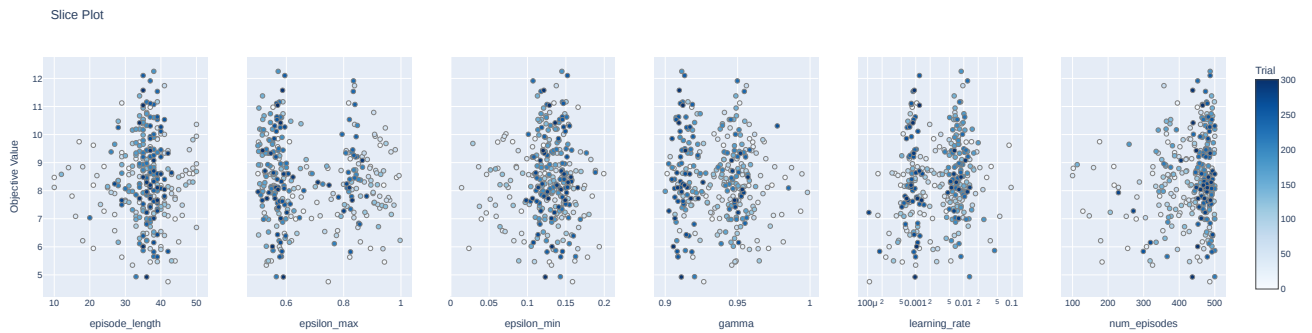


Figura 8: Resultados parámetros importantes Pareto

En la figura anterior se observan los valores conseguidos para los distintos parámetros más influyentes.

- **Episode length:** en esta ejecución, los valores de *episode length* que dan mejores resultados no se encuentran en el extremo como en el caso anterior, sino que permanecen entre 30-40.
- **Epsilon max:** los mejores resultados para la *epsilon max* se divide en dos distribuciones, una ligeramente por debajo de 0.6, y la otra aproximadamente 0.85.
- **Epsilon min:** los mejores resultados para la *epsilon min* se encuentran entre 0.1 y 0.15.
- **Num episodes:** cómo era de esperar la cantidad de episodios converge hacia el máximo del rango de valores posibles, permitiendo una mejor aproximación de la *Q-Table* para seleccionar las acciones según el criterio de optimalidad de Pareto de la forma más precisa posible.

4.2.3. Welfare

4.2.3.1 Distribución de recompensas

En la siguiente figura se puede observar cómo los distintos parámetros afectan a los valores de las recompensas resultantes.

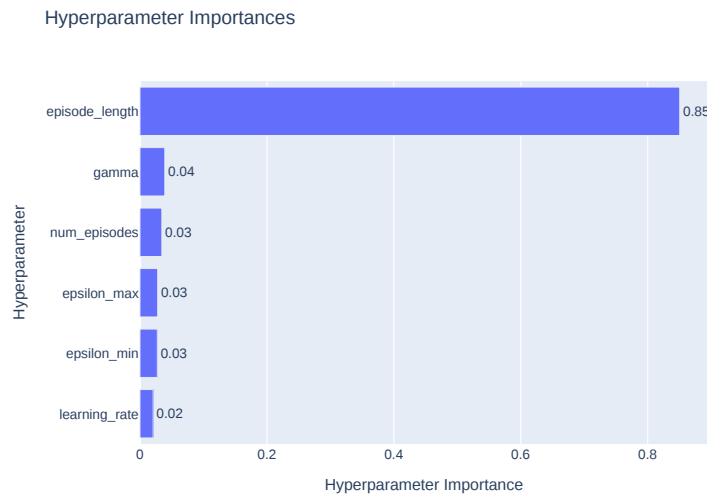


Figura 9: Parametros más importantes Welfare de *Optuna*

Se puede ver que el parámetro que más impacto tiene sobre la recompensa es la cantidad de pasos máximo por episodio (`episode_lenght`), esto creemos que se debe a que, como *Welfare* intenta maximizar la recompensa colectiva, si los agentes saben que pueden llegar a obtener alguna recompensa, entonces eventualmente aprenden a ir hacia ella. En un principio los agentes tienen su estado pero no tienen ningún tipo de incentivo a ir hacia, por ejemplo, la dirección de su objetivo porque no han estado allí y no saben que les otorgará una recompensa. Haciendo muchos pasos aleatorios, eventualmente los gentes saben que existe una recompensa y esta en dirección a su objetivo, por lo que empiezan a aprender a moverse hacia este.

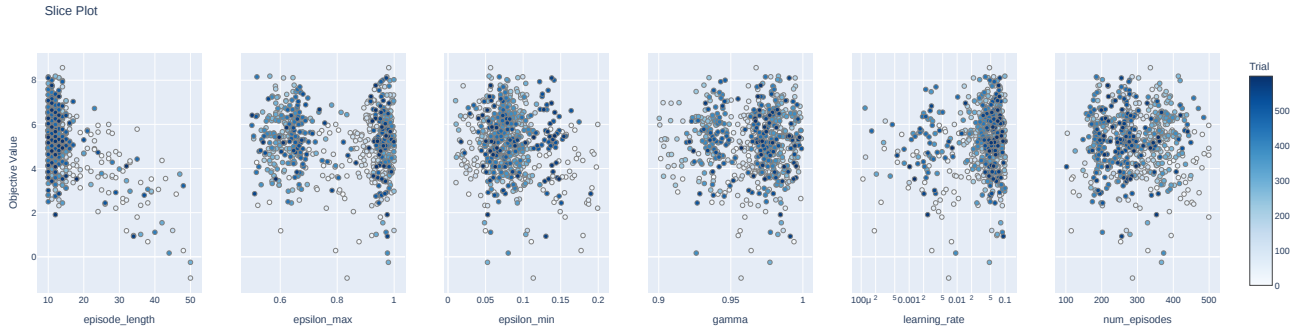


Figura 10: Resultados parametros importantes Welfare

- **Episode length:** Pese a la previa explicación, la cantidad de pasos máxima por episodio parece aproximarse al mínimo del rango en lugar de el máximo. Una explicación razonable es que, al limitar la cantidad de pasos de esta forma, los agentes aprenden a restringir la cantidad de pasos que pueden tomar hasta alcanzar el objetivo incluso en la evaluación. Optuna parece haber hallado el *threshold* al rededor de los 12 pasos por episodio, con valores inferiores se arriesga a que los agentes no encuentren jamás sus objetivos.
- **Epsilon Max:** Por las razones comentadas previamente, queda claro que una exploración altamente aleatoria beneficia a los agentes entrenados por criterio de *Welfare*.
- **Learning rate:** Está claro que, al minimizar los pasos por episodio, se debe maximizar la α , con tal de mantener la proporción de .aprendizaje.^{en} cada episodio.

El método demuestra ser bastante efectivo para 2 agentes independientemente de la complejidad del entorno, sin embargo, al pasar a más agentes, estos empiezan a tener un comportamiento errático. Observando varias ejecuciones con 2 agentes, exceptuando alguna ejecución donde se quedan haciendo movimientos cíclicos (probablemente por la representación del estado), este concepto de solución presenta un buen rendimiento.

4.2.4. Equilibrios de Nash

Busca estrategias donde ningún agente tiene incentivo para desviarse unilateralmente. Implementado como `NashSolutionConcept`, con probabilidades uniformes cuando hay múltiples equilibrios.

4.2.4.1 Distribución de recompensas

En el caso de los *Equilibrios de Nash* el hiperparámetro predominante ha resultado ser el número de episodios. Ésto no es sorprendente ya que la calidad del equilibrio encontrado depende enormemente de la aproximación de recompensas por acción a través de la Q-Table, ésta aproximación mejora con cada episodio, por lo que como veremos a continuación se intenta maximizar este hiperparámetro.

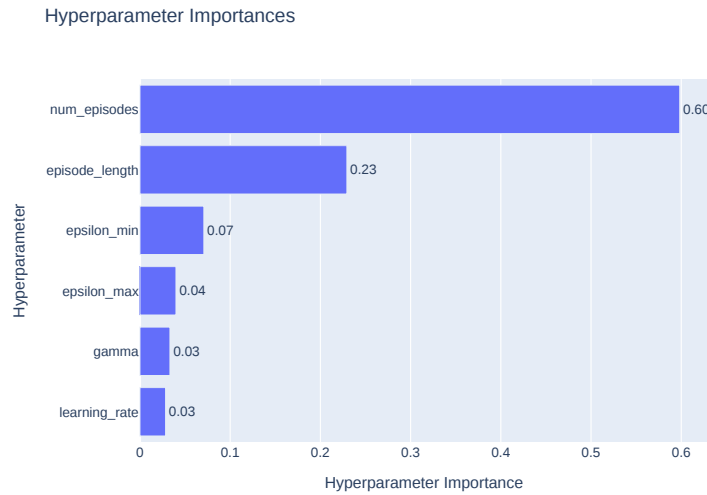


Figura 11: Parámetros más importantes Nash de *Optuna*

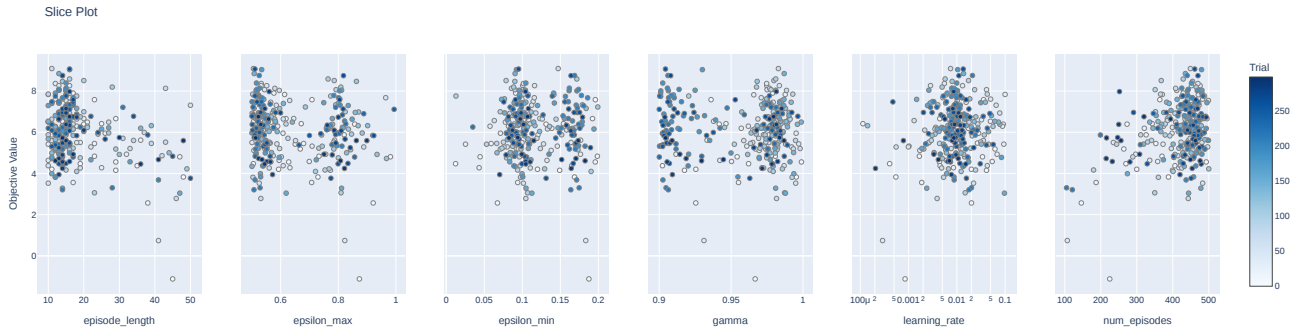


Figura 12: Resultados parámetros importantes Nash

- **Num episodes:** Como se menciona previamente, los mejores resultados se obtienen al aproximar con mayor precisión la *Q-Table*, lo cual se logra al aumentar la cantidad de episodios totales.
- **Learning rate:** Por la misma razón se observan valores razonablemente altos en α .

4.2.5. Otras métricas

Pese a (o a causa de) una recolección de datos profunda y varias iteraciones de representación de los datos, no hemos sido capaces de comentar todas las métricas exhaustivamente. Adjuntamos ciertas figuras, algunas de estilo *contouring* y un análisis general del tiempo de ejecución en JAL-GT.

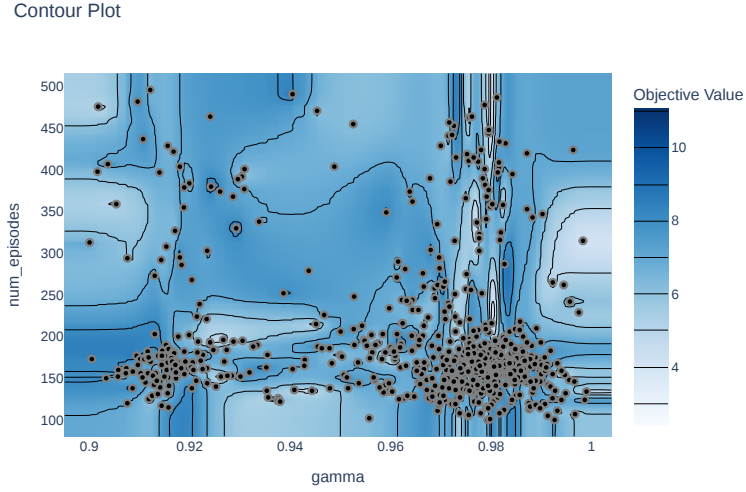


Figura 13: Gamma vs Num episodes - MiniMax concept JALGT

Este tipo de figuras permite observar claramente picos en espacios de pares de hiperparámetros, en este caso, como se menciona para MiniMax, queda claro que se deben ajustar la cantidad de episodios al rededor de 150. Además parece ser que hay dos franjas de valores para γ con mejores resultados que el resto. Hemos generado un gráfico de este estilo por cada par de hiperparámetros por combinación de algoritmo y concepto de solución, pensamos que este tipo de representación es ideal para guiarse en el espacio de hiperparámetros.

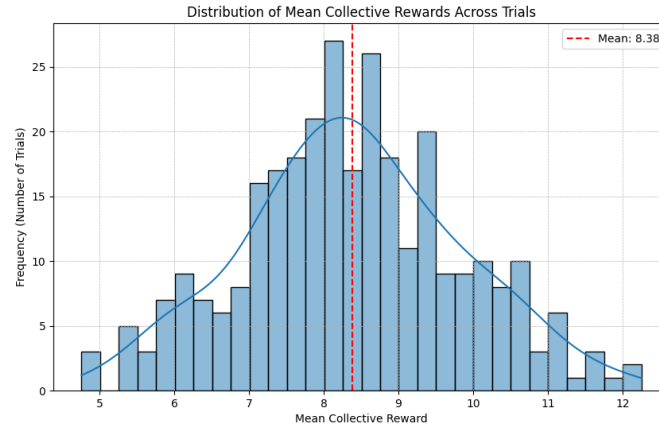


Figura 14: Caption

Como podemos observar, la distribución de recompensas en la evaluación de *trials* sigue una forma normal, ésto es de esperar teniendo en cuenta el tamaño de los datos.

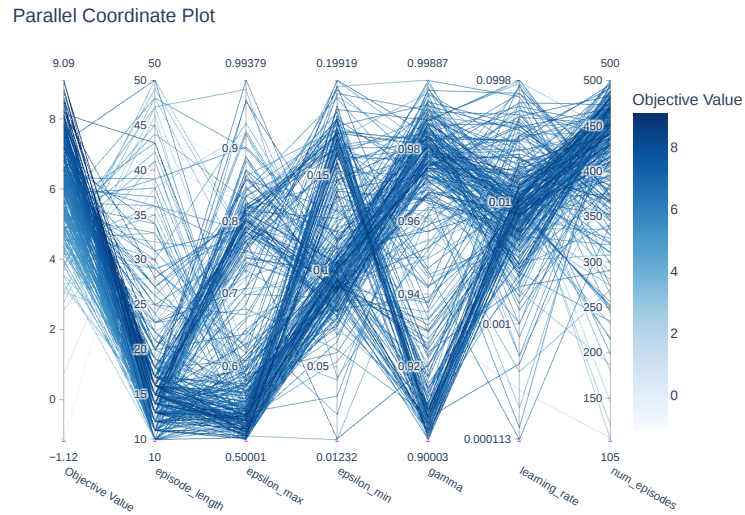


Figura 15: Coordenadas paralelas - Nash JALGT

Mediante esta representación tan peculiar podemos sacar conclusiones parecidas a las del apartado 4.2.4, por ejemplo el hecho de que maximizar la cantidad de pasos conlleva a una mayor recompensa colectiva.

Optimization History Plot

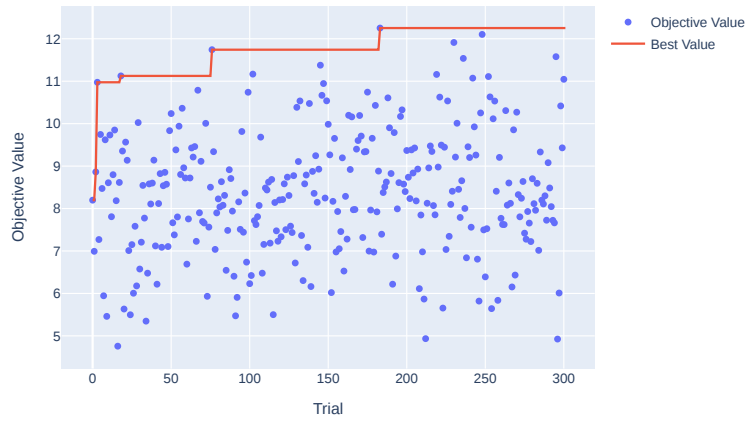


Figura 16: Historial de mejora para Pareto JALGT

En esta figura observamos los intentos de optimización de *Optuna*, parece que pese a ajustar los valores de hiperparámetros de forma teóricamente razonable la progresión no ha sido completamente lineal, pese a esto hay una clara mejora en cuanto al valor máximo conseguido alcanzando una media (de 4 samples) de más de 12 de recompensa colectiva.

El tiempo de ejecución, principalmente, dependía de 3 parámetros:

- **Número de episodios:** De forma lineal, la cantidad de episodios es proporcional al tiempo medio de ejecución.
- **Pasos máximos por episodios:** Este parámetro influencia el tiempo medio por episodio, aumentando de esta forma linealmente el tiempo medio de ejecución, ya que al permitir más pasos se ejecutan más instancias de `learn`, una de los métodos más caros en el código.
- **Concepto de solución:** Según nuestras observaciones, el concepto de solución de Pareto es mucho más lento que el resto, aproximadamente el triple. El resto de conceptos eran tan similares en tiempo de ejecución que los cambios se podrían culpar a la diferencia de *hardware* de las máquinas que hemos utilizado para recolectar datos.

4.3. IQL

Optimizamos IQL independientemente, sin conceptos de solución.

4.3.1. Distribución de recompensas

En la siguiente figura se puede observar cómo los distintos parámetros afectan a los valores de las recompensas resultantes. Los parámetros clave son el número de episodios, la cantidad de pasos máximos por episodio y la *learning rate*. Una importancia alta en α era de esperar tras observar los resultados de la práctica anterior, en la que era el valor predominante. Parece que en este caso, al ajustar mejor el rango de número de episodios, éste se ha vuelto más importante, quizás el valor máximo era demasiado bajo para IQL ya que los valores rápidamente convergen en la parte más alta. En cuanto a la cantidad máxima de pasos por episodio, limitar los pasos en el entrenamiento puede permitir minimizar los pasos hasta el objetivo, aplicando un efecto parecido al de la penalización por paso.

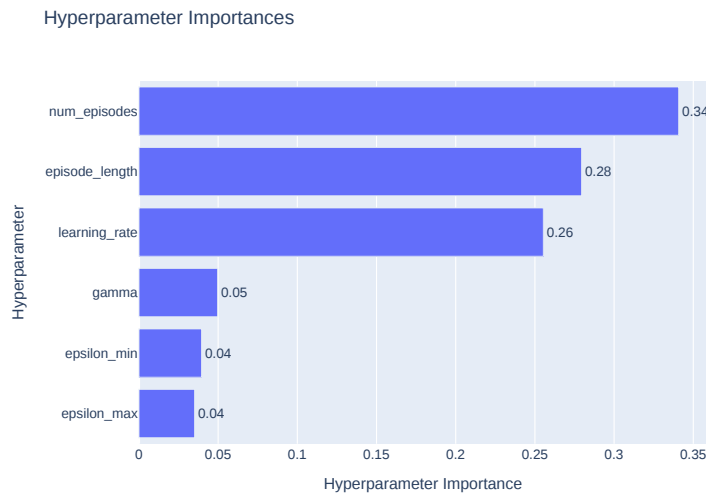


Figura 17: Parametros más importantes Nash de *Optuna*

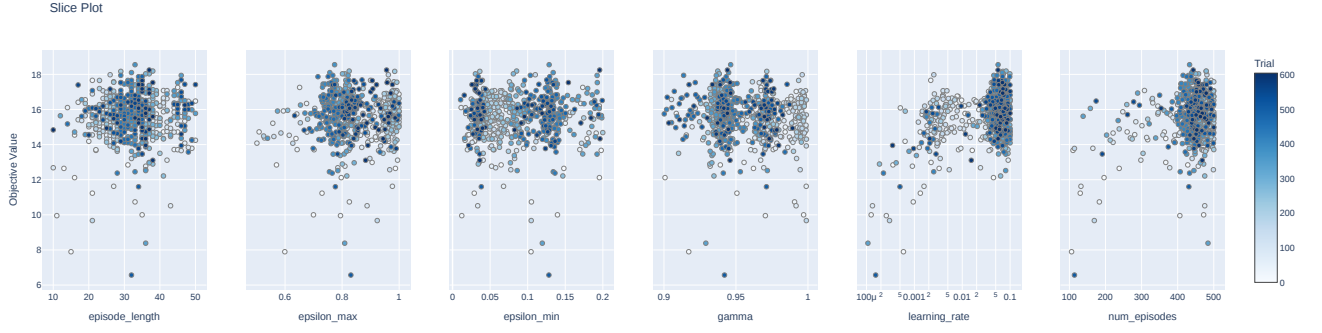


Figura 18: Resultados parámetros importantes Nash

- **Learning rate:** contrariamente a la práctica anterior, *IQL* parece haberse beneficiado de una mayor tasa de aprendizaje en este entorno, claramente mostrando una tendencia alcista para el parámetro.
- **Num episodes:** como se mencionó previamente, la cantidad de episodios es de gran importancia en la aproximación de la *Q-Table*, por lo que es natural que los valores converjan hacia el máximo del rango establecido.
- **Episode Length:** Optuna parece haber identificado, al rededor de los 35 pasos, un valor lo suficientemente alto cómo para favorecer la exploración del entorno sin pasarse para restringir los pasos en evaluación, un balance o *sweet spot* de la cantidad de pasos por episodio máximos.

Queda claro que *IQL* alcanza recompensas mucho mayores que JALGT clásico en la mayoría de los casos, de forma más consistente. Más sobre esto en las conclusiones.

4.3.2. Interpretación de las políticas

Las políticas resultantes con la mejor combinación de parámetros de *IQL* encontrados son las esperadas. Cada agente prioriza sus objetivos y en las muestras hechas no parece haber ningún tipo de colaboración, llegando a ver con relativa frecuencia puntos muertos donde dos agentes quieren ir a la posición del otro agente y acaban quedándose quietos. Un caso muy común en las ejecuciones parece ser en el que los agentes hacen movimientos cíclicos (de arriba a abajo todo el rato), esto se debe en gran parte a la codificación del estado. Como se ve en la figura siguiente, el agente rojo está atascado en un pasillo. Esto se debe a que intenta ir al punto que le deja más cerca de su objetivo en cada estado,

sin tener en cuenta ningún tipo de histórico, como este punto en la posición de abajo es inaccesible, no le queda de otra que subir pero, una vez arriba, el punto observable más cercano al objetivo es el de abajo.

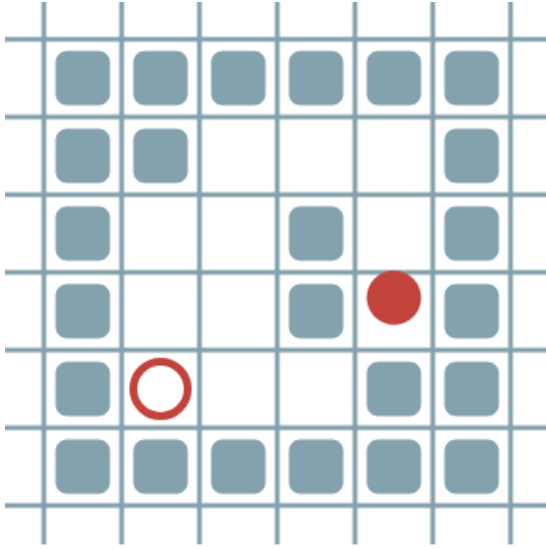


Figura 19

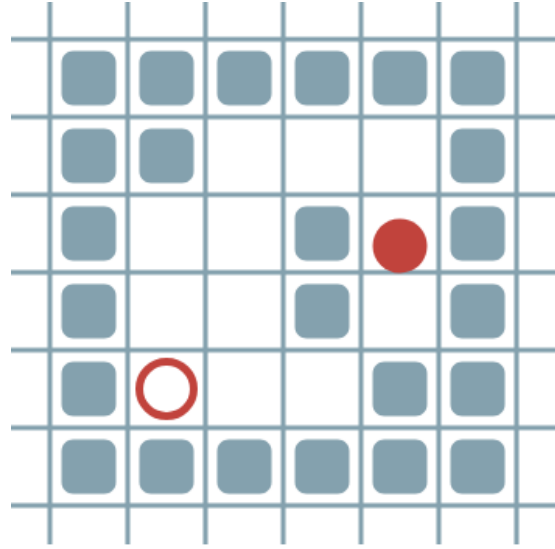


Figura 20

4.3.3. Generalización de los parámetros

Con la misma configuración de hiperparámetros del algoritmo cambiamos parámetros del entorno y vemos si estos son adecuados para aprender en entornos más complejos. Los parámetros del entorno probados han sido: size: 4, 6, 8 densidad de obstáculos: 0.0, 0.1, 0.2, 0.3 numero de agentes: 2, 3, 4

En nuestras ejecuciones hemos visto que, con una densidad de obstáculos de 0, independientemente del número de agentes y el tamaño del mapa, en su mayoría todos los agentes llegan a su destino, no ha sido en todos los casos debido a algún caso de estancamiento, donde los agentes intentan ir a donde hay otro agente.

Las recompensas tienden a caer bastante a medida que crece la densidad de obstáculos y el ratio entre la recompensa colectiva final respecto a una ejecución con otra con los mismos parámetros exceptuando el número de agentes es similar (ratio refiriendose a la recompensa obtenida entre la recompensa máxima obtenible, número de agentes * num mapas probados), por lo que la implementación de IQL es escalable en ese sentido.

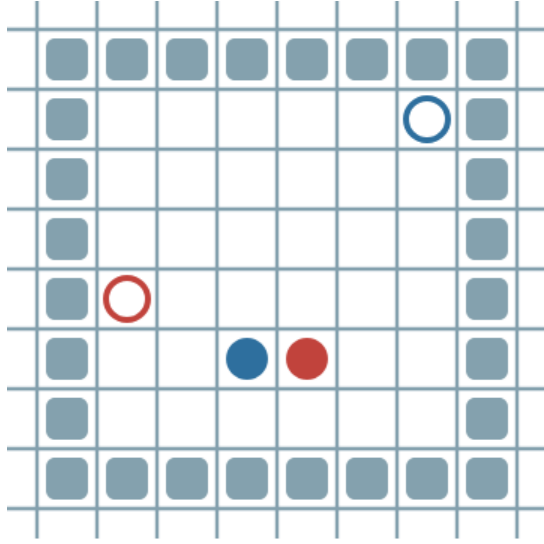


Figura 21

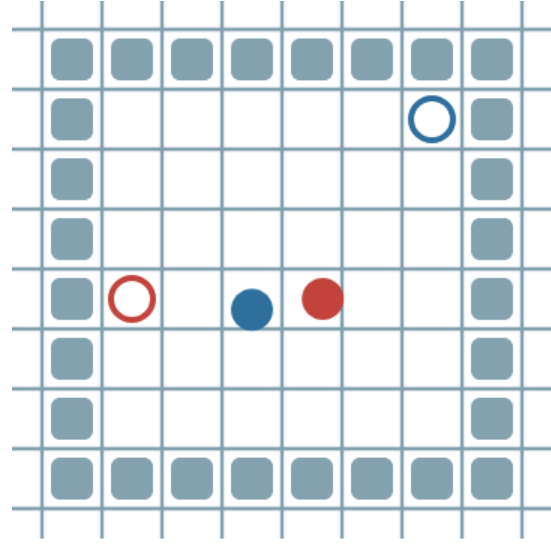


Figura 22: Estancamiento

4.4. JALGTNN

La implementación de JALGT con redes neuronales para estimar la función creemos que, en el entorno en el que estamos trabajando donde las recompensas son lineales y el estado discreto, no tiene ninguna ventaja sobre JALGT y además resulta computacionalmente más caro de ejecutar. Los resultados son similares a las pruebas hechas con JALGT con la diferencia de que los pasos máximos por episodio pierden bastante importancia en comparación con JALGT. Esta diferencia se debe, probablemente, al peso que se le atribuye a la recompensa por paso. Por la forma en que funcionan las redes, al hacer el gradiente con un learning rate muy bajo, el impacto que tienen las recompensas de los pasos futuros se diluye mucho mientras que en JALGT la recompensa por paso tiene un impacto constante.

4.4.1. Minimax

Este concepto de solución ha sido el menos estudiado en general debido a que solo hemos hecho pruebas con 2 agentes ya que no hemos generalizado el concepto de solución para un número indefinido de agentes. Demuestra unos resultados, en comparativa con el resto de métodos, bastante carentes. Los resultados son esperables debido a que minimax es un concepto de solución que no está orientado a la colaboración, si no a la competición, cosa que en este entorno en específico no nos interesa.

En el gráfico siguiente vemos que el parámetro con más importancia para el concepto de solución de Minimax son los pasos máximos por episodio.

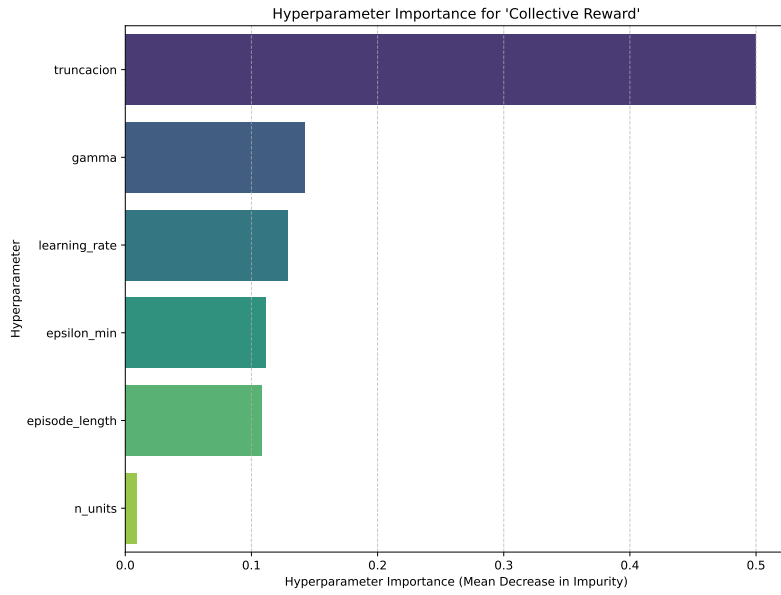


Figura 23: Parametros más importantes Minimax de *Optuna*

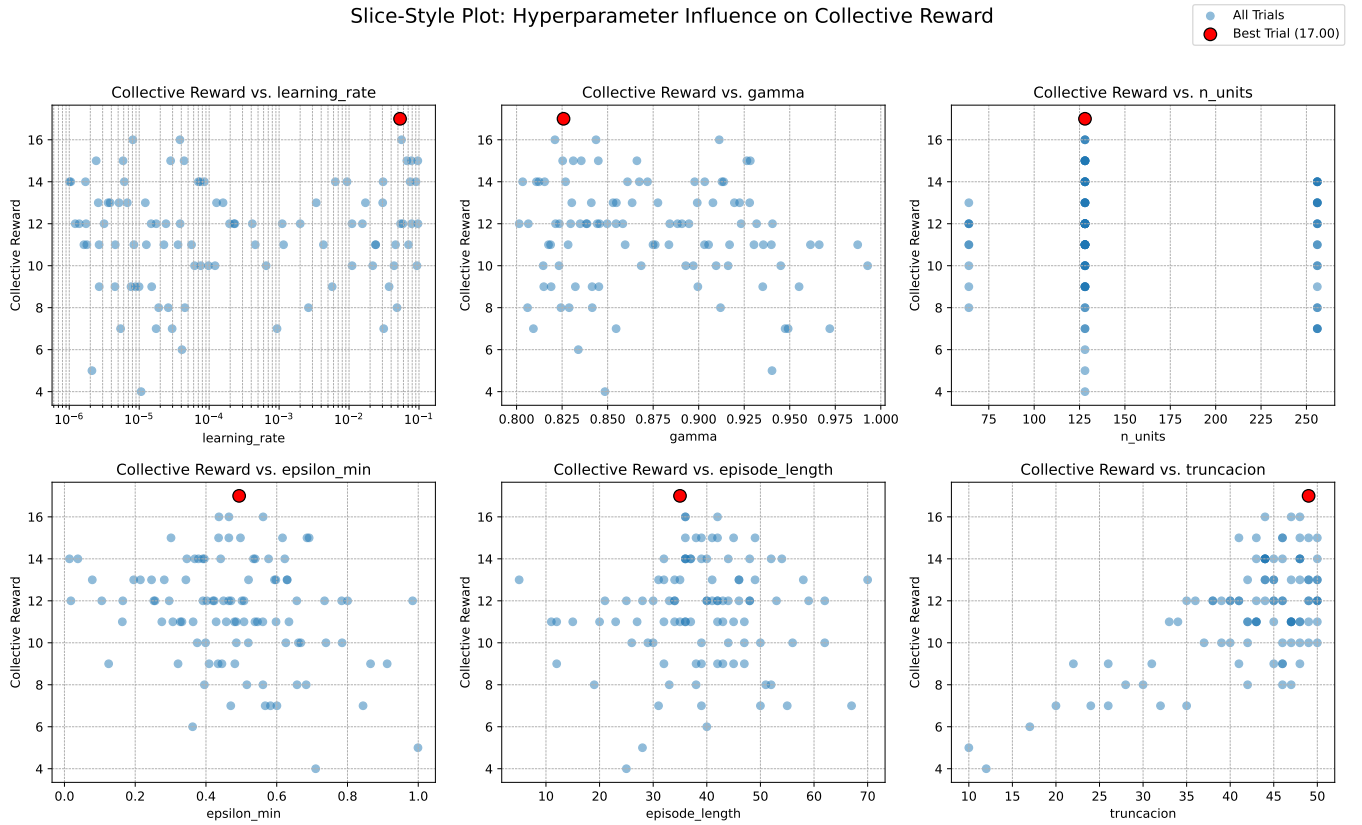


Figura 24: Resultados parámetros importantes Minimax

- **Truncacion:** en esta ejecución, los valores de *Truncacion* que dan mejores resultados se acercan mucho a los limites establecidos. Pero seguramente se deba a que si el

agente da suficientes pasos aleatorios, termina encontrando su objetivo.

- **Gamma:** los mejores resultados para la *Gamma* se encuentran sobre los 0.8-0.875.
- **Epsilon min:** los mejores resultados para la *epsilon min* se encuentran entre 0.0 y 0.4, indicando que, a diferencia que el concepto anterior, la mejor estrategia a seguir en el entrenamiento es la política aprendida por el momento.
- **Learning Rate:** los mejores resultados para el *Learning Rate* se concentran sobre 0.1.
- **Num episodes:** los mejores resultados para el *Num episodes* son unos 400 episodios ($10 \times \text{episode_length}$)
- **n_units:** los mejores resultados para las *n_units* han sido 128, indicando que 128 neuronas en la capa oculta son suficientes para estimar la función Q .

4.4.2. Óptimo de Pareto

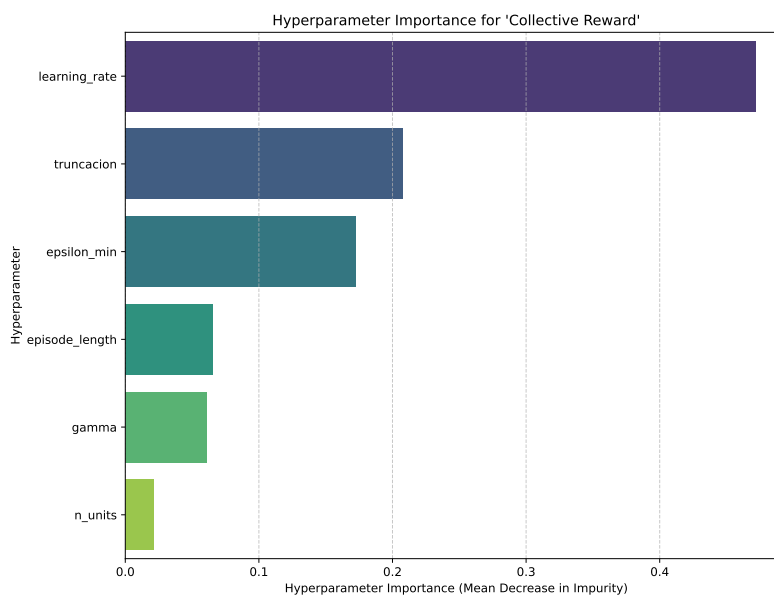


Figura 25: Parametros más importantes Pareto de *Optuna*

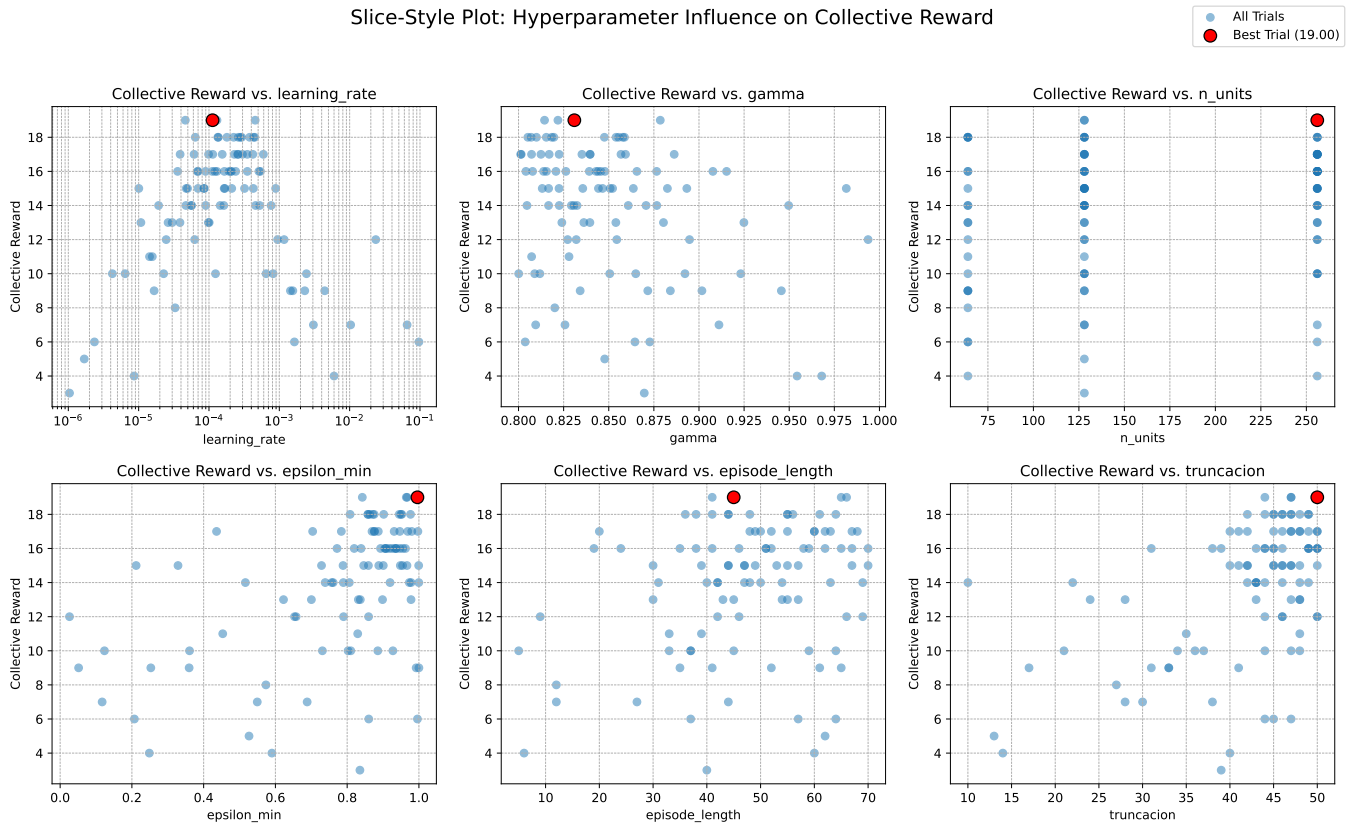


Figura 26: Resultados parámetros importantes Pareto

- **Truncacion:** en esta ejecución, los valores de *Truncacion* que dan mejores resultados se acercan mucho a los límites establecidos. Pero seguramente se deba a que si el agente da suficientes pasos aleatorios, termina encontrando su objetivo.
- **Gamma:** los mejores resultados para la *Gamma* se encuentran sobre los 0.8-0.875.
- **Epsilon min:** los mejores resultados para la *epsilon min* se encuentran entre 0.8 y 1, indicando que la mejor estrategia a seguir en el entrenamiento son movimientos aleatorios.
- **Learning Rate:** los mejores resultados para el *Learning Rate* se concentran sobre 0.0001.
- **Num episodes:** los mejores resultados para el *Num episodes* son unos 400 episodios ($10 \times \text{episode_length}$)
- **n_units:** los mejores resultados para las *n_units* han sido 256, aunque parece ser que tiene un comportamiento similar si no igual a 128 units, indicando que el número de

neuronas en la capa oculta no tiene porque ser tan grande, sobre todo en un entorno relativamente simple.

4.4.3. Welfare

En teoría el concepto de solución de bienestar debería maximizar la recompensa colectiva por lo que debería reducir los casos de estancamiento vistos en IQL, además de obtener de media recompensas mejores que el resto de conceptos de solución. Esto parece ser cierto para un número de 2 agentes donde en al menos dos ocasiones (de las políticas observadas haciendo el render), momento donde podría haber sucedido un estancamiento no pasaron y alguno de los dos agentes cede el paso al otro. Contrariamente a lo esperado, este concepto de solución ha sido el peor, solo por delante de minimax. Al igual que en JALGT, el método demuestra ser bastante efectivo para 2 agentes independientemente de la complejidad del entorno, sin embargo, al pasar a más agentes, estos empiezan a tener un comportamiento errático que tal vez se deba a nuestra implementación del concepto de solución a la hora de tratar con más de dos agentes. Observando varias ejecuciones con 2 agentes, exceptuando alguna ejecución donde se quedan haciendo movimientos cíclicos (probablemente por la representación del estado), este concepto de solución presenta un buen rendimiento.

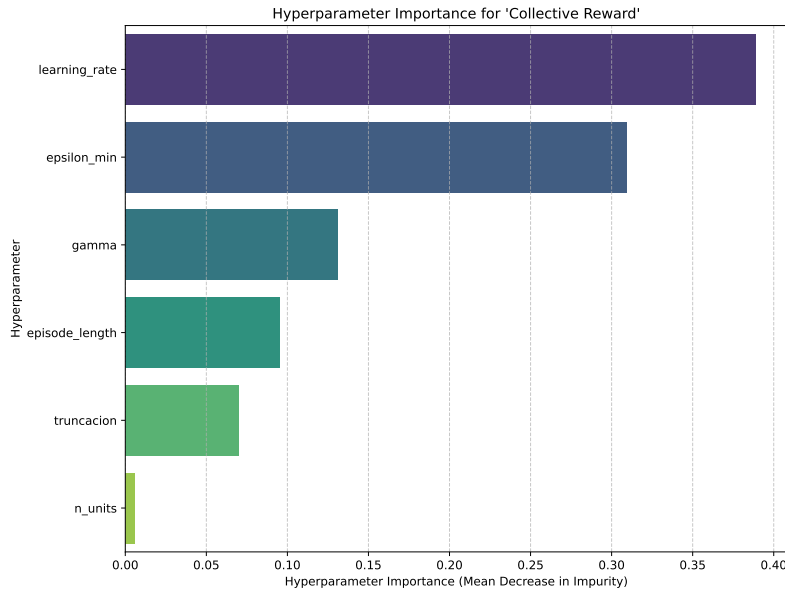


Figura 27: Parametros más importantes Welfare de *Optuna*

Slice-Style Plot: Hyperparameter Influence on Collective Reward

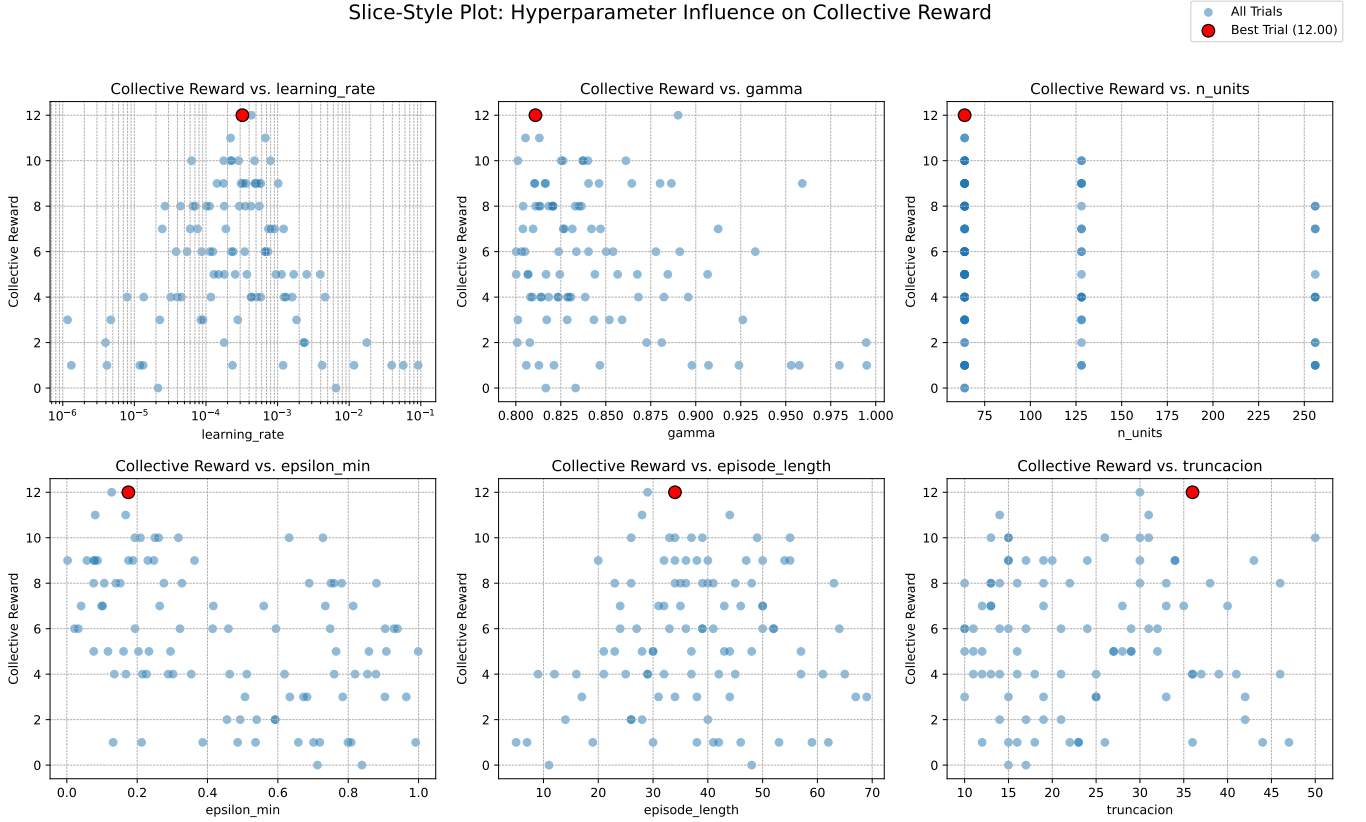


Figura 28: Resultados parámetros importantes Welfare

- **Truncacion:** en esta ejecución, los valores de *Truncacion* que dan mejores resultados se concentran por la parte inferior del rango disponible.
- **Gamma:** los mejores resultados para la *Gamma* se encuentran sobre los 0.8-0.875.
- **Epsilon min:** los mejores resultados para la *epsilon min* se encuentran entre 0.0 y 0.4, indicando que, a diferencia que el concepto anterior, la mejor estrategia a seguir en el entrenamiento es la política aprendida por el momento.
- **Learning Rate:** los mejores resultados para el *Learning Rate* se concentran sobre 0.0001.
- **Num episodes:** los mejores resultados para el *Num episodes* son unos 400 episodios ($10 \times \text{episode_length}$)
- **n_units:** los mejores resultados para las *n_units* han sido 64, el número de neuronas óptimas realmente no debería cambiar respecto al resto de conceptos de solución, pero es posible que Optuna pruebe antes con un número que le da buenos resultados y

empiece a crear un cierto bias hacia este valor, dejando sin probar el resto. Como con 64 units se muestra un resultado comparable al concepto anterior, puede significar que 64 neuronas en la capa oculta sean más que suficientes para estimar la función Q en este entorno.

4.4.4. Equilibrios de Nash

Mantiene la búsqueda de equilibrios pero con aproximación neuronal de los valores Q . En la teoría creemos que este concepto de solución debería ser el segundo mejor, solo por detrás del bienestar. Sin embargo, basándonos en nuestros resultados, vemos que nash si ha sido el segundo mejor método pero por detrás de Pareto, no de bienestar.

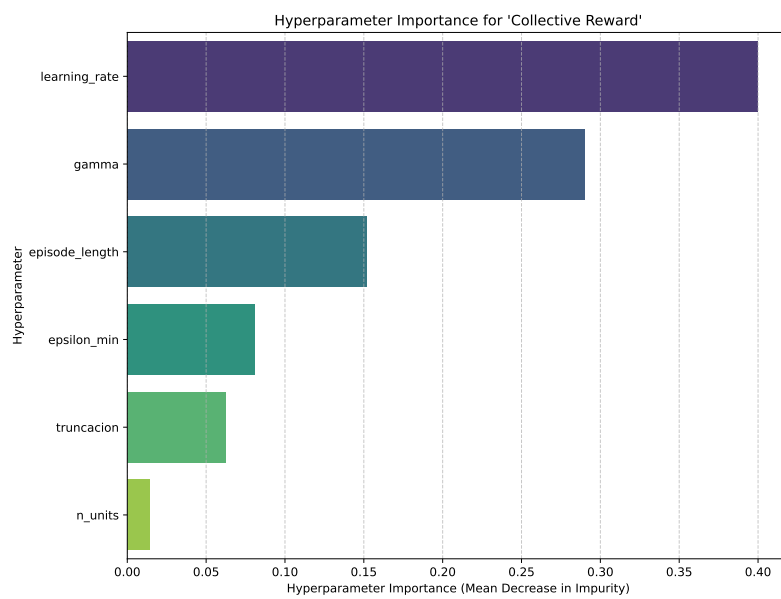


Figura 29: Parametros más importantes Nash de *Optuna*

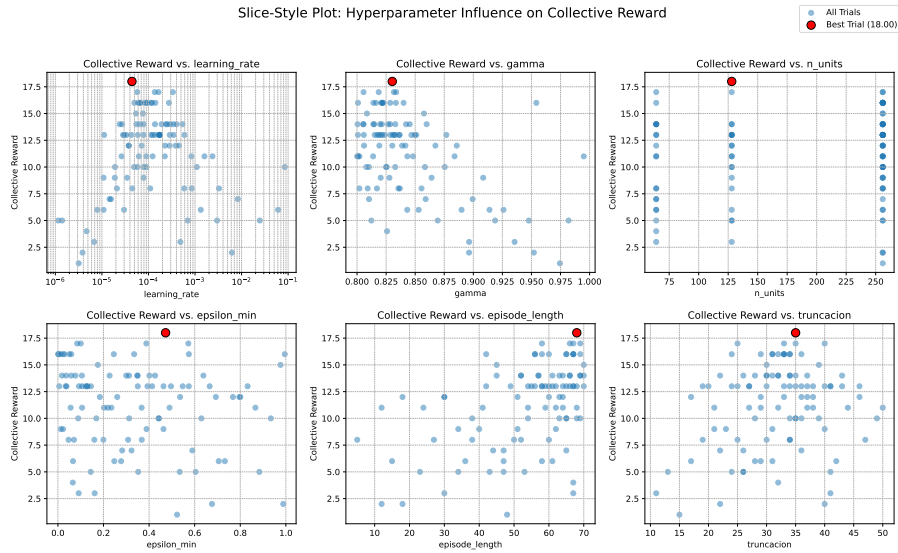


Figura 30: Resultados parámetros importantes Nash

- **Truncacion:** en esta ejecución, los valores de *Truncacion* rondan los 30 pasos.
- **Gamma:** los mejores resultados para la *Gamma* se encuentran sobre los 0.8-0.875.
- **Epsilon min:** los mejores resultados para la *epsilon min* se encuentran entre 0.2 y 0.6, aunque están bastante dispersos.
- **Learning Rate:** los mejores resultados para el *Learning Rate* se concentran sobre 0.0001.
- **Num episodes:** los mejores resultados para el *Num episodes* son unos 700 episodios (10*episode_length)
- **n_units:** los mejores resultados para las *n_units* han sido 128.

4.5. Parámetros óptimos

Después de la experimentación llevada a cabo, se ha llegado a la conclusión de que los mejores parámetros son proporcionados por el algoritmo JALGT-NN utilizando el concepto de solución de *Óptimos de Pareto* junto a los siguientes parámetros:

- **learning_rate:** 0.0001125029614709035
- **gamma:** 0.8309286705124114
- **n_units:** 128
- **epsilon_min:** 0.4724122440730355
- **episode_length:** 68
- **truncation:** 35

A diferencia de el resto de conceptos de solución, *Óptimos de Pareto* busca la mejora de todos sin que ningún agente empeore, cosa que fomenta estrategias y cooperación entre agentes. Además, es posible que la red neuronal haga una muy buena estimación de los valores de las Q-tables, hasta de estados no vistos, dando mejores resultados que su contraparte (JALGT).

5. Conclusión

A través de una optimización sistemática de hiperparámetros con Optuna, se ha intentado no solo identificar las configuraciones más eficaces, sino también obtener una profunda comprensión teórica de la interacción entre los algoritmos, los conceptos de solución y la naturaleza del problema.

Los resultados han corroborado las expectativas teóricas sobre el comportamiento de cada algoritmo. IQL, debido a su naturaleza descentralizada, aunque ha demostrado ser propenso a equilibrios subóptimos como estancamientos y movimientos cíclicos, ha alcanzado los mejores resultados de forma consistente tanto en valor de recompensas como en tiempo de ejecución. Por otro lado, JAL-GT, aunque teóricamente más robusto para la coordinación, ha dejado en evidencia sus limitaciones de escalabilidad inherentes a los métodos de aproximación de la tabla de recompensas. La variante JALGT-NN ha demostrado ser superior a la original, al combinar la capacidad de generalización de las redes neuronales con el formalismo de la teoría de juegos, poniéndose varias veces a la par, o incluso por encima de IQL.

El hallazgo más significativo de este estudio es que el hiperparámetro dominante depende fundamentalmente del concepto de solución empleado, lo que revela la lógica fundamental de cada uno. Para Minimax y Welfare, la longitud máxima del episodio (`episode_length`) ha sido el factor clave, ya que define el *threshold de solvencia* y una limitación de pasos máxima necesarias para que estos conceptos operen de forma significativa. En contraste, para el Equilibrio de Nash, el número de episodios fue predominante, subrayando la necesidad de un "presupuesto de negociación" temporal suficiente para que las políticas de los agentes converjan en un entorno no estacionario.

Finalmente, este trabajo concluye que la combinación del algoritmo JALGT-NN con el concepto de solución de Óptimo de Pareto ofrece el rendimiento más robusto y eficaz para el problema de coordinación en POGEMA aunque, debido a la *Curse of dimensionality*, IQL pasa a ser una mejor opción rápidamente al aumentar el número de agentes. Esta sinergia es exitosa porque el Óptimo de Pareto fomenta de manera inherente la cooperación y la búsqueda de mejoras mutuamente beneficiosas, mientras que la red neuronal proporciona la

escalabilidad y capacidad de generalización necesarias para aprender funciones Q precisas en espacios de estados complejos, superando incluso a su contraparte tabular en estados no vistos.

En caso de disponer de más tiempo hubiese sido interesante experimentar con los parámetros mencionados en el apartado [4.1](#) y realizar pruebas más específicas en cuanto a mezclas de conceptos de solución, tipos de mapas, cantidad de agentes e incluso otros tipos de algoritmos.